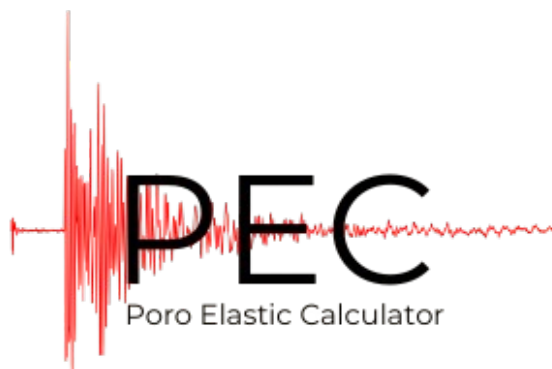




PoreElasticCalculator

Manual do Desenvolvedor

Versão 1.3

Autores:

Matheus Sousa Bastos (matheusbastos@lenep.uenf.br)

Nicolau Azevedo Prates (nicolauprates@lenep.uenf.br)

Outubro/2025

Sumário

1	Introdução	3
2	Funcionalidades Principais	3
3	Requisitos de Sistema	4
4	Instalação e Compilação	4
4.1	Método Padrão (CMake)	4
5	Estrutura do Projeto	5
6	Arquitetura do Software	6
6.1	Diagrama de Classes	6
6.2	Descrição das Classes Principais	7
7	Base de Dados de Minerais	7
8	Guia de Uso Detalhado	7
8.1	Opção 2: Carregar amostra de arquivo	7
8.2	Opção 3: Ver resumo das amostras	8
8.3	Opção 4 e 5: Geração de Gráficos	9
9	Guia para Extensões Futuras	10
9.1	Adicionando um Novo Modelo de Cálculo	10
9.2	Adicionando um Novo Tipo de Gráfico	10

1 Introdução

O software **PoreElasticCalculator** (PEC) foi desenvolvido em C++ como uma ferramenta robusta para o cálculo dos módulos elásticos efetivos — Módulo de Compressibilidade (K) e Módulo de Cisalhamento (G) — de amostras de rochas com diferentes composições mineralógicas. Utilizando as médias de Voigt, Reuss e Hill, o programa oferece uma análise precisa dos limites teóricos e da estimativa empírica das propriedades da rocha.

O objetivo deste manual é fornecer um guia denso e completo para desenvolvedores que desejam entender, manter e estender a funcionalidade do PEC. O documento detalha a arquitetura do software, a estrutura do projeto, os processos de compilação multiplataforma e as convenções de código adotadas.

2 Funcionalidades Principais

O programa oferece um conjunto completo de funcionalidades através de uma interface de linha de comando interativa:

- Inserção manual de dados de amostras diretamente no terminal.
- Carregamento em lote de múltiplas amostras a partir de arquivos de texto (`.txt`) com formato estruturado.
- Cálculo automático dos módulos elásticos K e G para os limites de Voigt, Reuss e a média de Hill.
- Geração de um sumário tabular com as propriedades de entrada e os resultados calculados para todas as amostras.
- Criação de gráficos de alta qualidade utilizando o `gnuplot` para a visualização de dados, incluindo:
 - Perfis de K e G em função da profundidade.
 - Perfis de K e G em função da porosidade.
 - Análise de sensibilidade para a variação da proporção entre dois minerais.
- Configuração em tempo de execução para habilitar ou desabilitar a exibição dos gráficos na tela, enquanto o salvamento em arquivo `.png` é sempre garantido em um diretório de saída configurável.
- Nomenclatura única para arquivos de gráficos baseada em timestamp, prevenindo a sobrescrita de resultados.

3 Requisitos de Sistema

Para a compilação e execução correta do programa, o ambiente do desenvolvedor deve atender aos seguintes requisitos:

- **Sistema Operacional:** Windows 10/11, Linux (e.g., Ubuntu 20.04+) ou macOS. O projeto é totalmente multiplataforma.
- **Compilador C++:** Um compilador moderno com suporte ao padrão C++17 ou superior.
 - **Windows:** MSVC (incluso no Visual Studio Build Tools) ou MinGW-w64 (g++).
 - **Linux:** g++ ≥ 9.0 .
 - **macOS:** Clang (Xcode Command Line Tools).
- **CMake:** Sistema de automação de compilação, versão ≥ 3.15 . Essencial para a compilação do projeto.
- **Gnuplot:** Software para a geração de gráficos. Deve estar instalado e com seu diretório executável adicionado à variável de ambiente **PATH** do sistema.

4 Instalação e Compilação

O projeto utiliza CMake para gerenciar o processo de compilação, garantindo portabilidade e simplicidade. O método recomendado é o uso do CMake.

4.1 Método Padrão (CMake)

1. **Clone o Repositório:** Obtenha o código-fonte do projeto.
2. **Instale os Pré-requisitos:** Certifique-se de que o compilador C++, CMake e Gnuplot estão instalados e acessíveis no **PATH** do sistema.
3. **Crie o Diretório de Build:** A partir da raiz do projeto (PEC_v3), execute:

```
1 mkdir build
2 cd build
3
```

Listing 1: Criando o diretório de build

4. **Configure o Projeto:** A estrutura de compilação é criada com base no seu sistema operacional.

- **Para Linux/macOS ou Windows com MinGW/g++:**

```

1 # Para Linux ou macOS:
2 cmake ..
3
4 # Para Windows usando g++ (MinGW):
5 cmake -G "MinGW Makefiles" ..
6

```

Listing 2: Configurando com CMake (MinGW/Linux/macOS)

- **Para Windows com Visual Studio (MSVC):** Abra o **Developer Command Prompt for VS 2022** e execute:

```

1 cmake ..
2

```

Listing 3: Configurando com CMake (MSVC)

5. **Compile o Código-fonte:** Ainda no diretório build, execute:

```

1 cmake --build .
2

```

Listing 4: Compilando o projeto

6. **Resultado:** O executável `PoreElasticCalculator` (ou `PoreElasticCalculator.exe` no Windows) será criado dentro da pasta build.

5 Estrutura do Projeto

O projeto é organizado em uma estrutura de diretórios clara e lógica para separar o código-fonte, os cabeçalhos, os dados e os arquivos de build.

```

PEC_v3/
|
+-- build/                // (Criado pelo dev) Arquivos de compilacao e executavel
+-- database/             // Arquivos de dados, como a base de minerais
|   +-- minerais.csv
|
+-- include/              // Arquivos de cabecalho (.h) das classes
|   +-- CAmostra.h
|   +-- CMineral.h
|   ... (outras classes)
|
+-- src/                  // Arquivos de implementacao (.cpp)

```

```

|   +-- CAmostra.cpp
|   +-- CMineral.cpp
|   ... (outras classes)
|
+-- test/                // Arquivos de entrada para testes e saida de plots
|   +-- multiplasAmostras.txt
|   +-- Plots/           // (Criado pelo app) Graficos gerados
|
+-- CMakeLists.txt       // Script principal de configuracao do CMake
+-- main.cpp             // Ponto de entrada da aplicacao

```

6 Arquitetura do Software

O software é projetado com base nos princípios da Programação Orientada a Objetos (OOP), com cada classe possuindo uma responsabilidade única e bem definida.

6.1 Diagrama de Classes

O diagrama abaixo ilustra as principais classes e suas interações. A classe `CSimulador` atua como o orquestrador principal da aplicação.

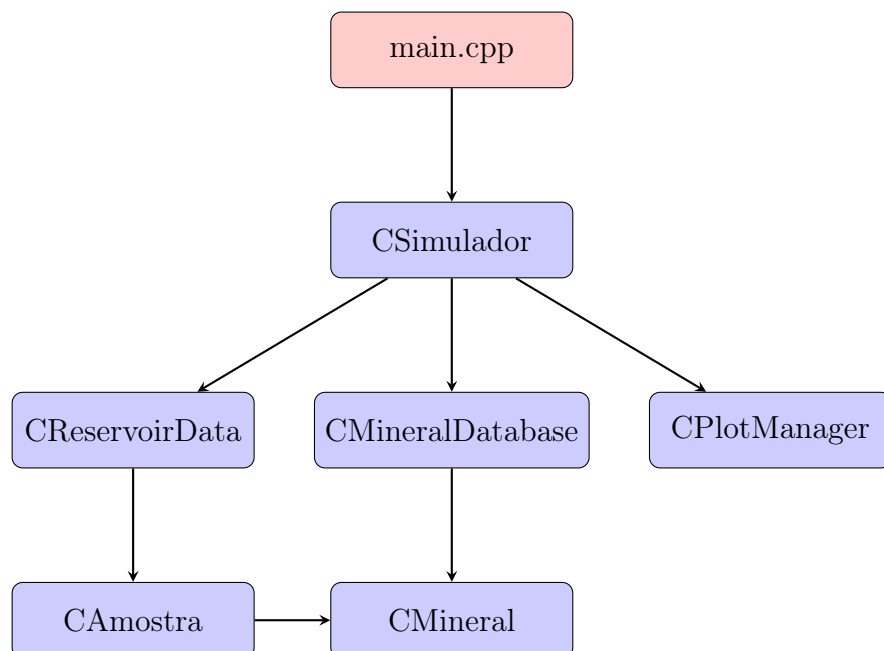


Figura 1: Diagrama simplificado da arquitetura de classes.

6.2 Descrição das Classes Principais

- **CSimulador**: A classe central que gerencia o fluxo da aplicação. É responsável por exibir o menu, processar a entrada do usuário e coordenar as ações entre as outras classes.
- **CReservoirData**: Container de dados que armazena e gerencia a lista de todas as amostras carregadas na sessão.
- **CAmostra**: Modela uma única amostra de rocha, contendo suas propriedades como nome, profundidade, porosidade e a composição mineralógica.
- **CMineralDatabase**: Responsável por carregar e fornecer acesso aos dados elásticos dos minerais a partir do arquivo `minerais.csv`.
- **CMineral**: Representa um mineral individual com seus módulos K e G.
- **CPlotManager**: Encapsula toda a lógica para a geração de gráficos. Interage com o `gnuplot` para criar scripts, salvar os gráficos em arquivos `.png` e exibi-los na tela.
- **CElasticCalculator**: Classe (e suas derivadas `CBulkCalculator`, `CShearCalculator`) que contém a lógica matemática para calcular os módulos elásticos efetivos usando as médias de Voigt, Reuss e Hill.

7 Base de Dados de Minerais

A flexibilidade do programa é garantida por uma base de dados externa. Segue abaixo a lista completa dos minerais presentes no nosso banco de dados padrão.

Para estender a base, basta editar o arquivo `database/minerais.csv`, adicionando novas linhas no formato: `nome_mineral,valor_K,valor_G`.

8 Guia de Uso Detalhado

Para iniciar o programa, execute o arquivo gerado na compilação. O menu interativo principal será exibido.

8.1 Opção 2: Carregar amostra de arquivo

Permite o carregamento em lote. O arquivo deve seguir um formato estruturado, onde cada amostra é um bloco de dados.

```

1 reservatorio: Bacia_Campos
2 amostra: Amostra_A1
3 profundidade: 2500
4 porosidade: 15.2
5
6 quartzo 70
7 albita 20
8 kerogenio 10
9
10 reservatorio: Bacia_Campos
11 amostra: Amostra_A2
12 profundidade: 2550
13 porosidade: 14.8
14
15 quartzo 65
16 albita 25
17 kerogenio 10

```

Listing 5: Exemplo de formato para o arquivo de entrada ('input.txt')

8.2 Opção 3: Ver resumo das amostras

Exibe uma tabela consolidada no console com as informações de todas as amostras carregadas e os resultados dos módulos calculados.

```

5. Gerar gráfico de variação mineralógica
6. Configuracao: Exibir graficos na tela (Atual: Sim)
7. Sair
Escolha: 3

```

Nome	Profundidade (m)	Porosidade (%)	Compressibilidade K (GPa)	Cisalhamento G (GPa)
Amostra_1	1000.00	10.00	37.00	44.00
Amostra_2	1050.00	10.50	33.53	38.03
Amostra_3	1100.00	11.00	31.11	34.14
Amostra_4	1150.00	11.50	29.32	31.32
Amostra_5	1200.00	12.00	27.95	29.12
Amostra_6	1250.00	12.50	26.84	27.30
Amostra_7	1300.00	13.00	25.94	25.74
Amostra_8	1350.00	13.50	25.18	24.37
Amostra_9	1400.00	14.00	24.53	23.13
Amostra_10	1450.00	14.50	23.97	21.99

```

--- MENU ---
1. Inserir nova amostra manualmente
2. Carregar amostra de arquivo
3. Ver resumo das amostras
4. Gerar gráficos: profundidade e porosidade
5. Gerar gráfico de variação mineralógica
6. Configuracao: Exibir graficos na tela (Atual: Sim)
7. Sair
Escolha:

```

Figura 2: Exemplo da tabela com o resumo das amostras.

8.3 Opção 4 e 5: Geração de Gráficos

Gera e salva os gráficos na pasta `test/Plots/`. Se a exibição estiver ativa, as janelas são abertas sequencialmente e fechadas ao pressionar Enter no terminal.

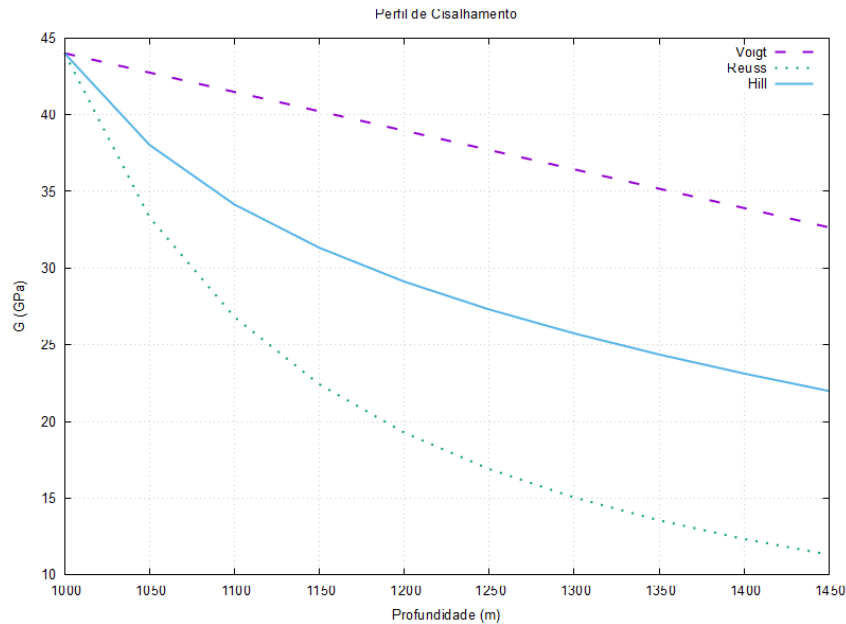


Figura 3: Exemplo de gráfico gerado para profundidade.

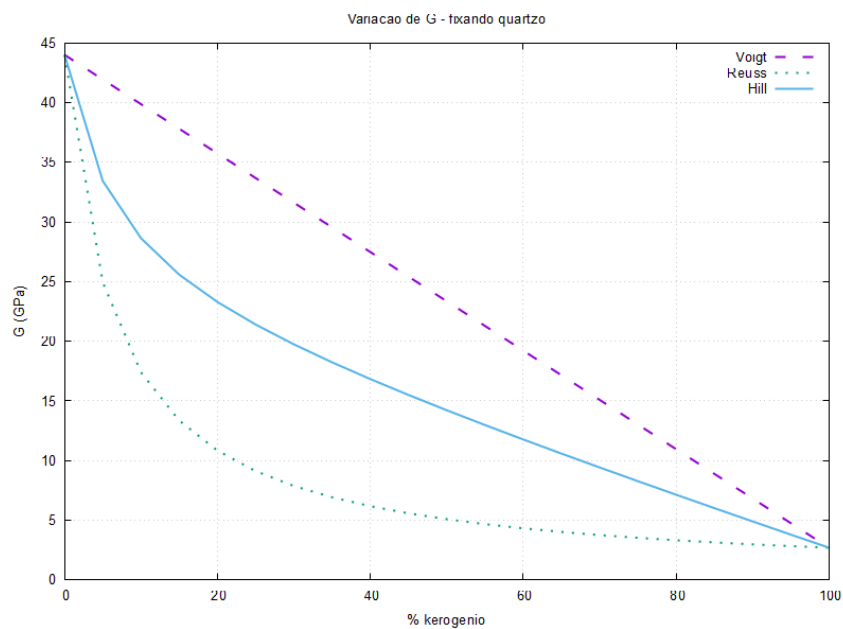


Figura 4: Exemplo de gráfico gerado para variação mineralógica.

9 Guia para Extensões Futuras

A arquitetura modular do PEC foi projetada para facilitar a adição de novas funcionalidades.

9.1 Adicionando um Novo Modelo de Cálculo

Para implementar um novo método de cálculo de módulo elástico (e.g., Média de Hashin-Shtrikman):

1. Crie uma nova função dentro da classe `CElasticCalculator` (ou suas derivadas) que implemente a nova equação.
2. Adicione um campo ao struct de resultados em `CReservoirData.h` para armazenar o novo valor.
3. Modifique a função `CReservoirData::PrintSummary()` em `CReservoirData.cpp` para exibir o novo resultado na tabela de resumo.
4. Atualize a classe `CPlotManager` para incluir a nova curva nos gráficos, se desejado.

9.2 Adicionando um Novo Tipo de Gráfico

Para criar um novo tipo de gráfico:

1. Adicione uma nova opção no menu da classe `CSimulador`.
2. Crie uma nova função pública na classe `CPlotManager` (e.g., `PlotNovaAnalise(...)`).
3. Dentro desta nova função, implemente a lógica para extrair os dados necessários de `CReservoirData` e chame a função privada `GeneratePlotScript` com os parâmetros apropriados.